

浙江大学实验报告

姓名：_____ 专业：_____ 学号：_____

课程名称：_____ 信息与电子工程导论 _____ 任课老师：_____

实验名称：_____ 基于 MATLAB 的信号频谱分析 _____ 实验日期：_____

1 实验目的和要求

1.1 实验目的

- (1) 初步认识 MATLAB 的使用。
- (2) 了解如何在 MATLAB 中使用 FFT, IFFT 等方式进行信号频谱分析。
- (3) 尝试构建一个信号，并从时域和频域进行分析，并进行一定的信号处理。

1.2 实验要求

- (1) 构建一个信号，至少含有 3 个频率分量，为周期信号或非周期信号。
- (2) 利用 MATLAB 分析其频谱。
- (3) 利用 MATLAB 对频谱作适当的处理，观察信号的变化。

2 实验原理

计算机只能处理离散有限长信号，因此需将连续信号采样离散化。傅里叶变换建立了时域与频域的联系，对于离散信号的变换，只有离散傅立叶变换（DFT）才能被适用，对于其它的变换类型只有在数学演算中才能用到。快速傅里叶变换（FFT）是 DFT 的高效算法，逆变换（IFFT）可将频域恢复为时域。利用 FFT 和 IFFT 可对信号进行频域处理（如滤波）并观察时域变化。

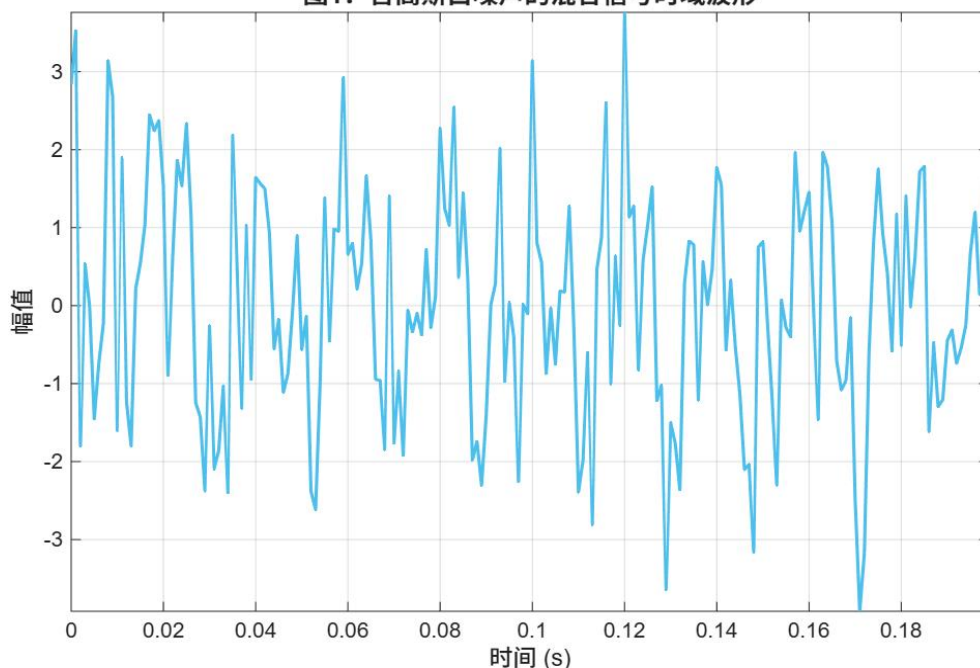
3 实验内容

- (1) 构建一个含有三个频率分量的信号，并添加随机白噪声。
- (2) 在时域中绘制含噪信号。
- (3) 对含噪信号进行快速傅里叶变换，在频域中绘制信号得到频谱。
- (4) 对含噪信号进行滤波处理，与原信号和参考信号进行比较。
- (5) 将采样点数增加，观察分辨率变化情况。

4 实验结果和分析

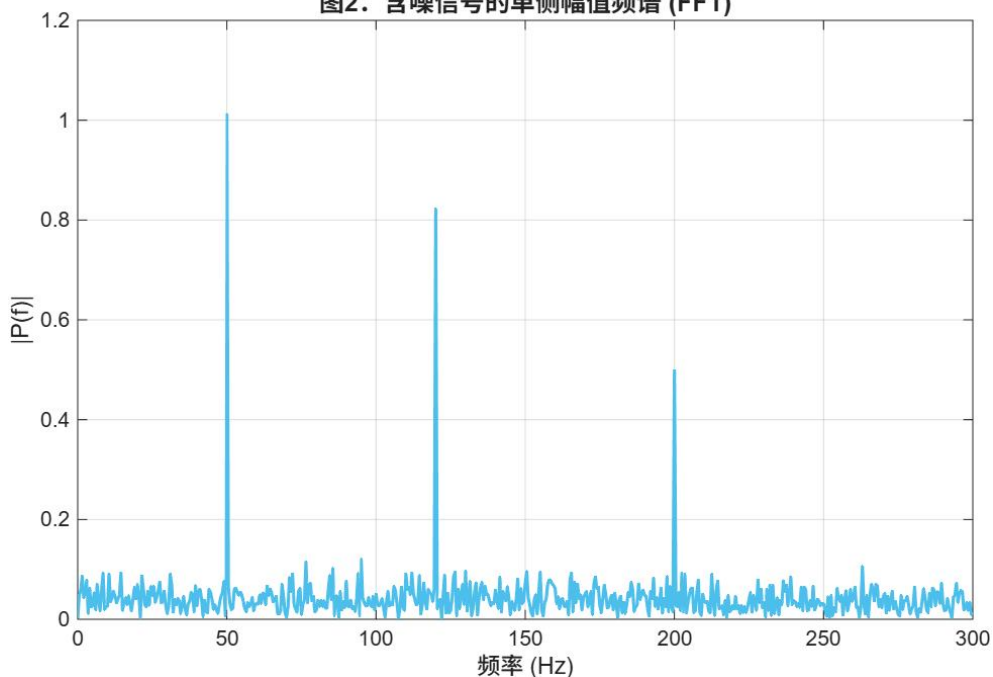
我们构建了由一个幅值为 1.0 的 50Hz 正弦信号、一个幅值为 0.8 的 120Hz 正弦信号以及一个幅值为 0.5 的 200Hz 正弦信号三个分量构成的信号，添加高斯白噪声以模拟真实的含噪信号，然后通过 MATLAB 得到了它的时域表示。（如图 1）

图1：含高斯白噪声的混合信号时域波形



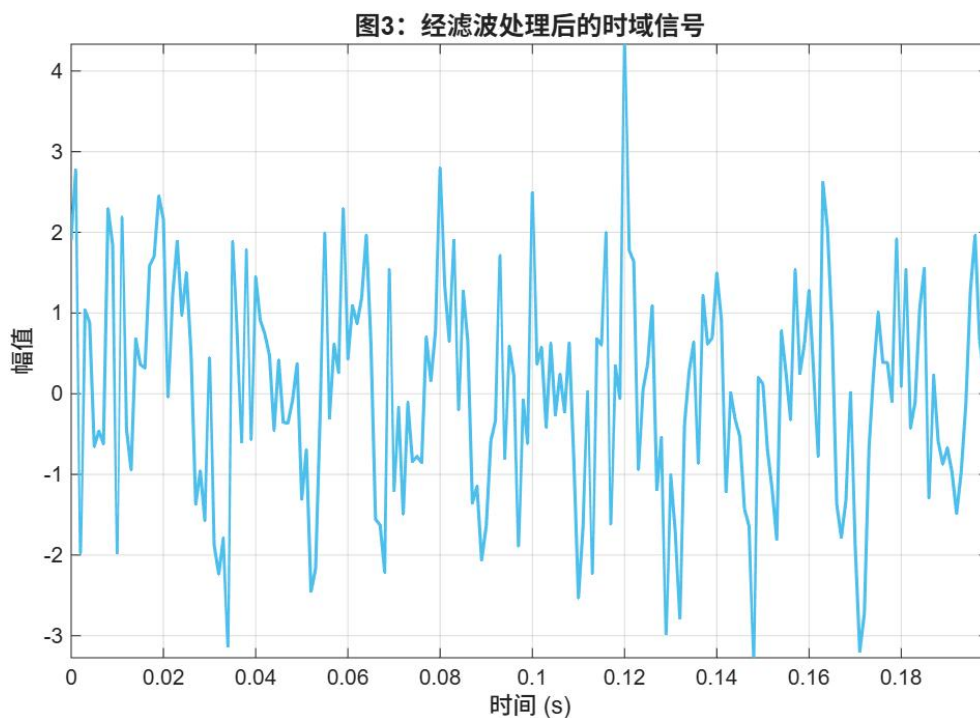
然后，我们对含噪信号进行快速傅里叶变换（FFT），并绘制其单侧频谱。（如图2）

图2：含噪信号的单侧幅值频谱 (FFT)

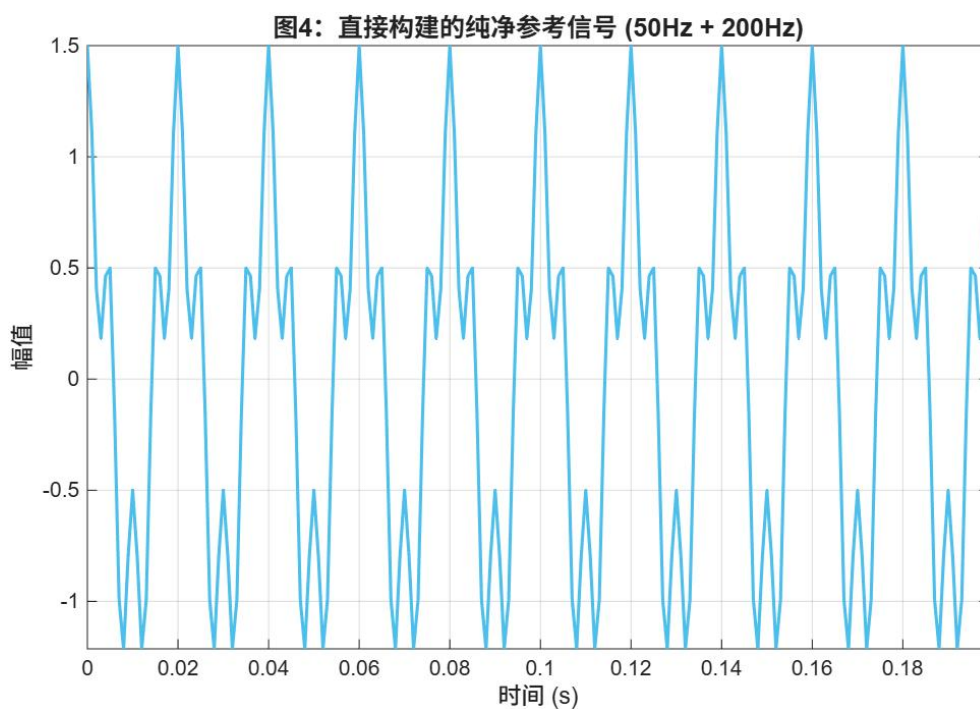


通过 FFT 变换，信号从时域转为频域。可以发现，频谱图中清晰地出现了三个峰值，分别位于 50Hz、120Hz 和 200Hz，且幅值比例约为 1.0 : 0.8 : 0.5，这与实验设定的参数完全吻合。频谱下方的“基底”抬升是由高斯白噪声引起的。

接下来，我们对含噪信号进行滤波处理，消除 80-150Hz 频段（含 120Hz 特征分量），并利用傅里叶逆变换（IFFT）得到了经滤波处理后的信号的时域表示。（如图3）

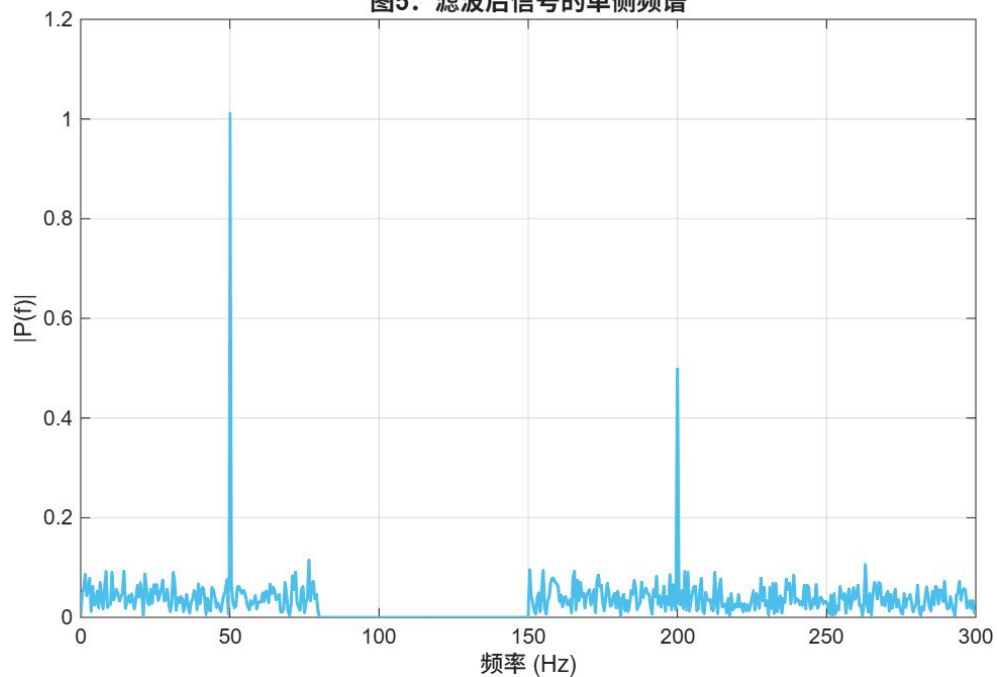


为直观得出滤波操作的作用效果，我们再次构建了一个由幅值为 1.0 的 50Hz 正弦信号和一个幅值为 0.5 的 200Hz 正弦信号构成的无噪声标准信号，并得到其时域表示。（如图 4）



可以看到，滤波后的含噪信号与标准信号的时域表示仍旧有较大差异，这是由于高斯白噪声本身分布在全频段，滤波后 0-80Hz 以及 150Hz 以上频率的噪声依然存在，将其傅里叶变换后清晰可见。（如图 5）

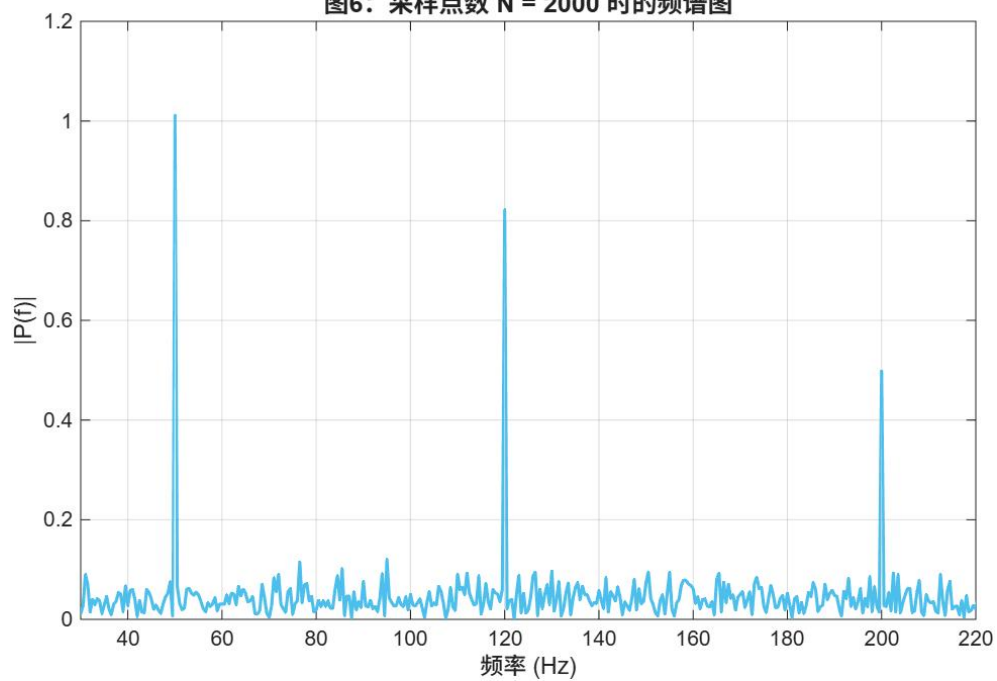
图5：滤波后信号的单侧频谱

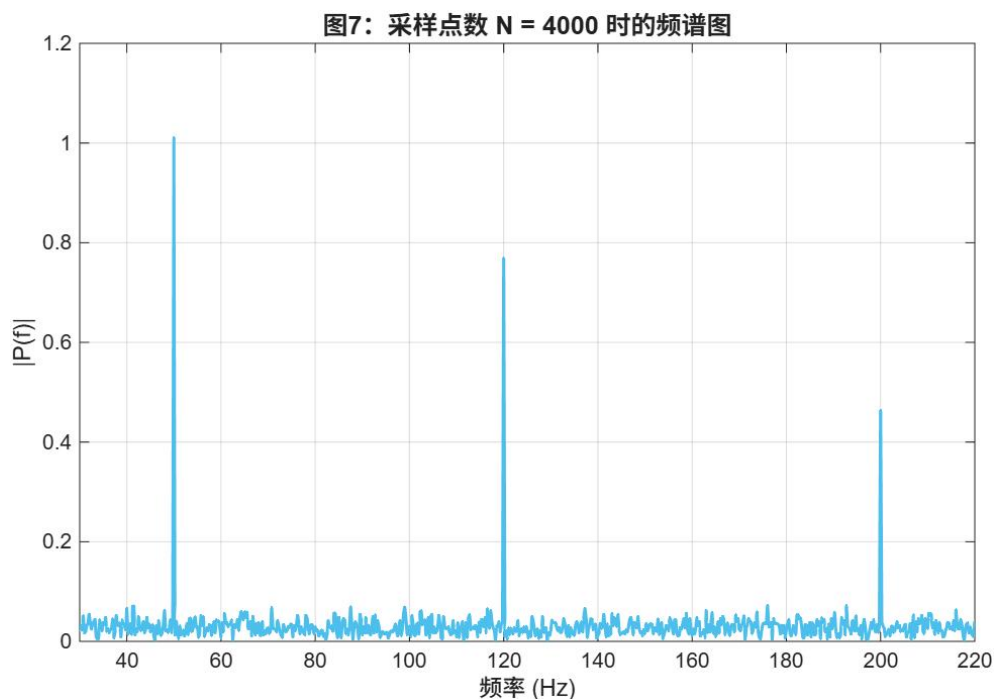


但图 3 与图 1 相比之下，我们可以认为滤波操作达到了预期结果。

接下来，我们改变了采样点数分别为 $N=2000$ 及 $N=4000$ ，得到了如下（图 6、图 7）的结果。

图6：采样点数 $N = 2000$ 时的频谱图





5 实验结论

通过对图 1 至图 5 的观察可以得出：通过快速傅里叶变换，我们可以得到信号的频谱，在 MATLAB 中可以通过令指定范围内的信号幅值等于 0 的方法，来起到滤波的作用。尽管我们可以通过滤波在一定程度上还原原始信号，由于噪声干扰具有高度的随机性，完全消除噪声在物理上是难以实现的。因此，在信号传输过程中，必须充分评估随机干扰带来的负面影响。在信号构建阶段，应当增大不同信号特征间的区分度，以增强系统的抗干扰能力，从而最大限度地降低随机噪声对信息传输准确性的冲击。

同时，由图 6、图 7 可知：改变信号的绘制点数，在采样频率不变时，图像分辨率随点数增多而提高。

6 源代码与分析

```
% 参数初始化
Fs = 1000;           % 采样频率
T = 1/Fs;            % 采样周期
N = 2000;            % 采样点数
t = (0:N-1)*T;       % 时间轴
light_blue = [0.30, 0.75, 0.93];

%% 1. 含噪信号时域波形 (图1)
s = 1.0*cos(2*pi*50*t) + 0.8*cos(2*pi*120*t) + 0.5*cos(2*pi*200*t);
rng('default');
y = s + randn(size(t));

figure(1);
plot(t(1:200), y(1:200), 'Linewidth', 1.5, 'Color', light_blue);
title('图1: 含高斯白噪声的混合信号时域波形', 'FontSize', 12);
xlabel('时间 (s)'); ylabel('幅值'); grid on; axis tight;

%% 2. FFT单侧频谱绘制 (图2)
Y = fft(y);
P2 = abs(Y/N);
P1 = P2(1:N/2+1);
P1(2:end-1) = 2*P1(2:end-1);
f = Fs*(0:(N/2))/N;
```

```

%% 3. 滤波处理: 消除 80-150Hz 频带 (图3)
Y_filtered = Y;
f_start = 80; f_end = 150;
idx_start = round(f_start * N / Fs) + 1;
idx_end = round(f_end * N / Fs) + 1;

% 频域置零处理
Y_filtered(idx_start:idx_end) = 0;
Y_filtered(N-idx_end+2 : N-idx_start+2) = 0;

% IFFT 还原回时域
y_filtered = ifft(Y_filtered, 'symmetric');
figure(3);
plot(t(1:200), y_filtered(1:200), 'Linewidth', 1.5, 'Color', light_blue);
title('图3: 经滤波处理后的时域信号', 'FontSize', 12);
xlabel('时间 (s)'); ylabel('幅值'); grid on; axis tight;

%% 4. 直接构建的纯净对比波形 (图4)
S_ref = 1.0*cos(2*pi*50*t) + 0.5*cos(2*pi*200*t);
figure(4);
plot(t(1:200), S_ref(1:200), 'Linewidth', 1.5, 'Color', light_blue);
title('图4: 直接构建的纯净参考信号', 'FontSize', 12);
xlabel('时间 (s)'); ylabel('幅值'); grid on; axis tight;

%% 5. 滤波后的频谱图 (图5)
P2_filt = abs(Y_filtered/N);
P1_filt = P2_filt(1:N/2+1);
P1_filt(2:end-1) = 2*P1_filt(2:end-1);

figure(5);
plot(f, P1_filt, 'Linewidth', 1.5, 'Color', light_blue);
title('图5: 滤波后信号的单侧频谱', 'FontSize', 12);
xlabel('频率 (Hz)'); ylabel('|P(f)|'); xlim([0 300]); grid on;

```

```

%% 6. 采样点数 N=2000 的频谱 (图6)
figure(6);
plot(f, P1, 'Linewidth', 1.5, 'Color', light_blue);
title('图6: 采样点数 N = 2000 时的频谱图', 'FontSize', 12);
xlabel('频率 (Hz)'); ylabel('|P(f)|'); xlim([30 220]); grid on;

%% 7. 采样点数 N=4000 的频谱 (图7)
N2 = 4000; t2 = (0:N2-1)*T;
y2 = 1.0*cos(2*pi*50*t2) + 0.8*cos(2*pi*120*t2) + 0.5*cos(2*pi*200*t2) + randn(size(t2));
Y2 = fft(y2); P1_2 = abs(Y2/N2); P1_2 = P1_2(1:N2/2+1); P1_2(2:end-1) = 2*P1_2(2:end-1);
f2 = Fs*(0:(N2/2))/N2;

figure(7);
plot(f2, P1_2, 'Linewidth', 1.5, 'Color', light_blue);
title('图7: 采样点数 N = 4000 时的频谱图', 'FontSize', 12);
xlabel('频率 (Hz)'); ylabel('|P(f)|'); xlim([30 220]); grid on;

```